



Instituto Infnet

Análise e Segurança de Agentes de IA: Projeto de Bloco

Etapa 5 e 6

Professor: Tiago Cariolano de Souza Xavier

Email : tiago.xavier@prof.infnet.edu.br

Definição de Modelo Regressor Linear



Regressão Linear

- Técnica para modelagem e análise de dados numéricos cujo objetivo é encontrar uma relação linear entre os dados
 - Assume que duas ou mais variáveis possuem uma relação linear
- Considere a expressão:

$$y_i = \beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i} + \cdots + \beta_k x_{ki}$$

- y_i valor previsto para o exemplo k
 - Variável dependente (variável-alvo)
- $x_{1i}, x_{2i}, \dots, x_{ki}$ atributos do exemplo k
 - Cada atributo corresponde a uma dimensão no espaço multidimensional
- β_0 : intercepto
- $\beta_1, \dots, \beta_k$: coeficientes que indicam como cada atributo influencia a previsão
 - Cada coeficiente representa o efeito de um atributo sobre a previsão, mantendo os demais constantes

Regressão Linear Simples

- **Regressão Linear Simples:** considera apenas uma variável independente (atributo) para explicar uma variável dependente (target)
 - Representado em duas dimensões (o atributo no eixo-x e o target no eixo-y):

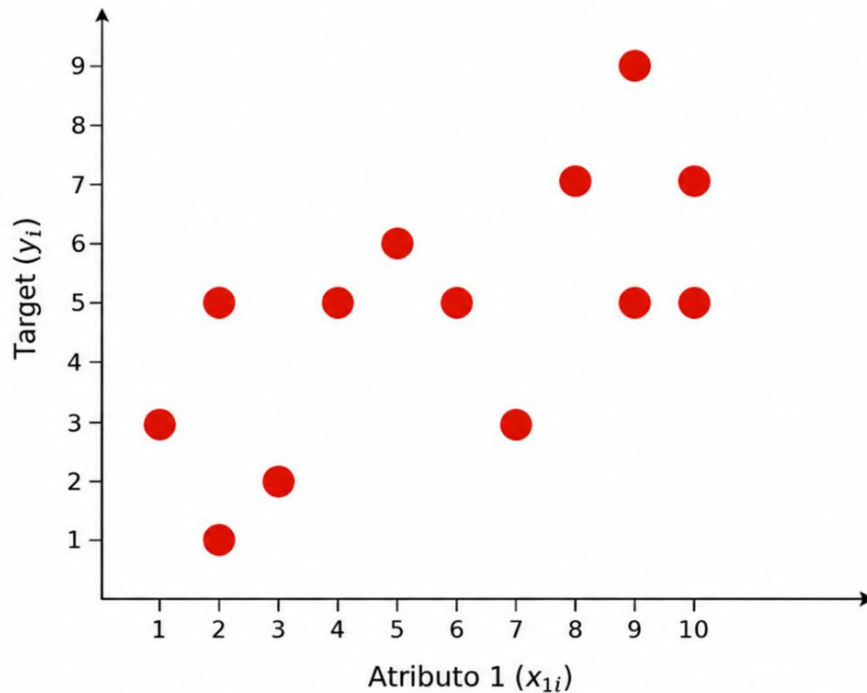
$$y_i = \beta_0 + \beta_1 x_{i1}$$

- y_i : variável dependente (target) do exemplo i
- x_{i1} : valor do atributo para o exemplo i
- β_0 : intercepto — valor de (y) quando $x=0$
- β_1 : coeficiente angular — indica quanto (y) varia para cada aumento de uma unidade em x

Regressão Linear Simples

i	x_{1i}	y_i
1	1	3
2	2	1
3	2	5
4	3	2
5	4	5
6	5	6
7	6	5
8	7	3
9	8	7
10	9	5
11	9	9
12	10	5
13	10	7

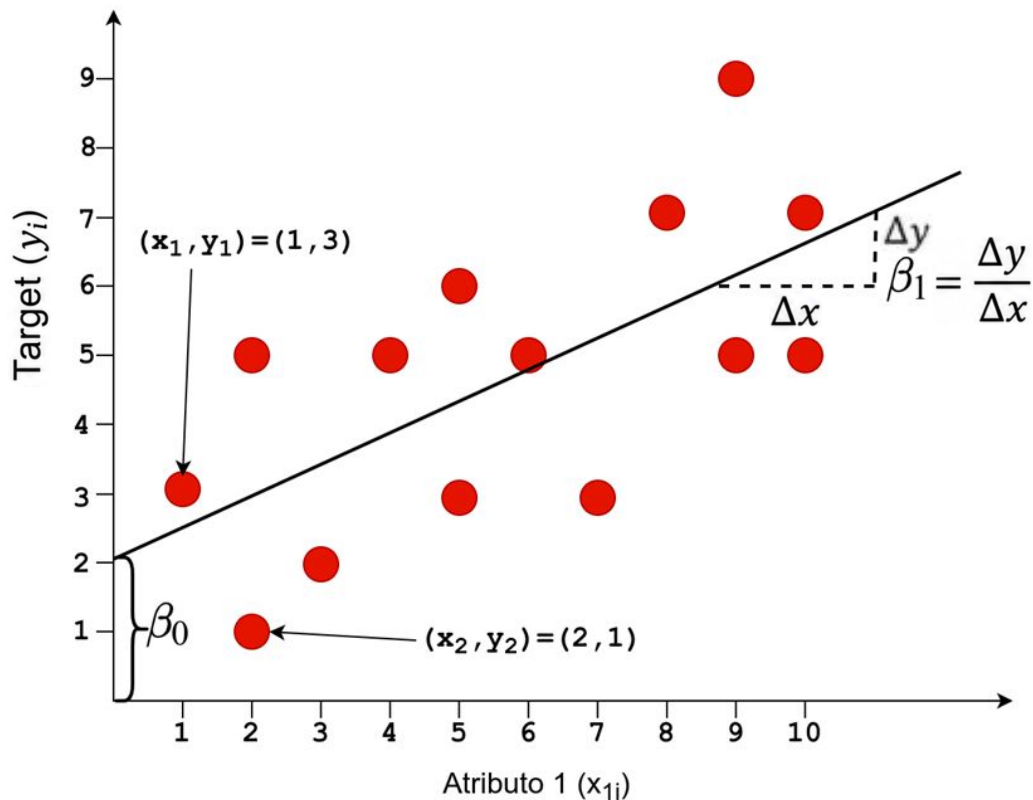
Conjunto de Dados



Regressão Linear Simples

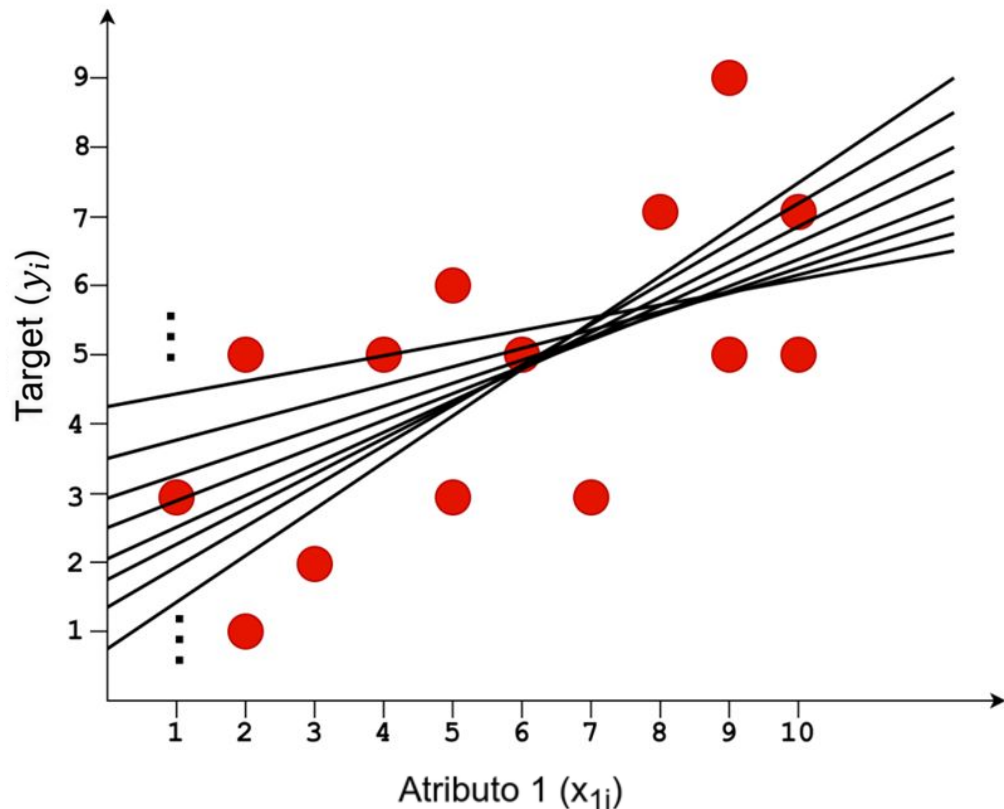
i	x_{1i}	y_i
1	1	3
2	2	1
3	2	5
4	3	2
5	4	5
6	5	6
7	6	5
8	7	3
9	8	7
10	9	5
11	9	9
12	10	5
13	10	7

Conjunto de Dados



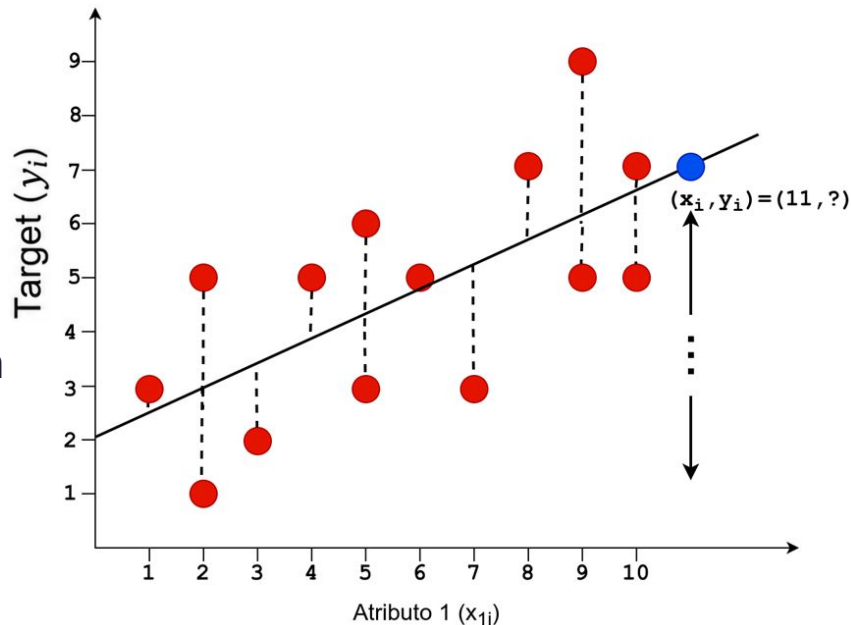
Regressão Linear Simples

- Selecionar diferentes valores de β_0 e β_1 gera diferentes retas
- Determinar a melhor reta consiste em encontrar os melhores valores de β_0 e β_1
- **Como determinar os melhores valores para β_0 e β_1 ?**



Regressão Linear

- **Selecionar a reta (β_0 e β_1) que minimiza os erros entre os pontos**
 - Cálculo dos erros:
 - Erro Absoluto Médio
 - Erro Quadrático Médio
- **Previsão:**
 - Dado um novo valor de x_i , além dos valores de β_0 e β_1 , basta aplicar a equação da reta para descobrir y_i



Regressão Linear no sklearn

- **Importar a biblioteca:**
 - `from sklearn import linear_model`
- **Instanciar Regressor Linear:**
 - `rl = linear_model.LinearRegression ()`
 - `rl`: regressor linear instanciado
- **Construir e treinar o modelo regressor linear:**
 - `modelo = rl.fit (atributos, target)`
 - `modelo`: modelo do regressor linear
 - `atributos`: atributos do conjunto de dados
 - `target`: valores de target do conjunto de dados

Regressão Linear no sklearn

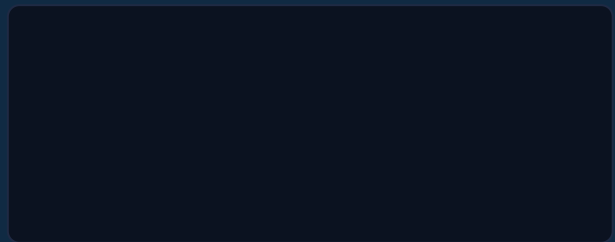
- Realizar previsões:

- `previstos = modelo.predict (atributos_teste)`
 - `atributos_teste`: Atributos do conjunto de teste
 - `previstos`: previsões dos valores de target (y_i)

- Obter coeficientes:

- `modelo.coef_`: vetor contendo os coeficientes $\beta_1, \beta_2, \dots, \beta_k$, um para cada atributo
- `modelo.intercept_`: intercepto, que corresponde a β_0
- Para regressão linear simples:
 - `modelo.coef_[0]` corresponde a β_1
 - `modelo.intercept_` corresponde a β_0

Avaliação de Modelo e Análise Residual



Avaliação do Regressor

- **Erro absoluto médio (Mean Absolute Error - MAE):** calcula a média das diferenças entre os valores previstos e os valores reais

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

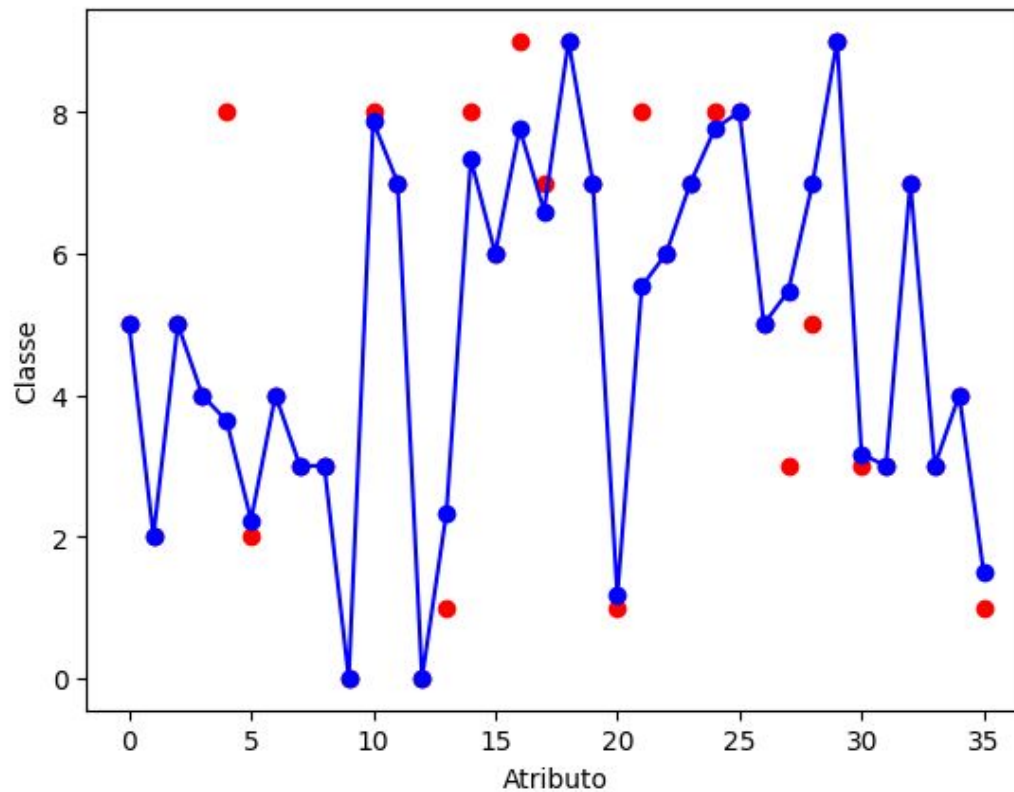
- **Erro quadrático médio (Mean Squared Error - MSE):** calcula a média das diferenças quadráticas entre os valores previstos e os valores reais

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Valor real na posição i

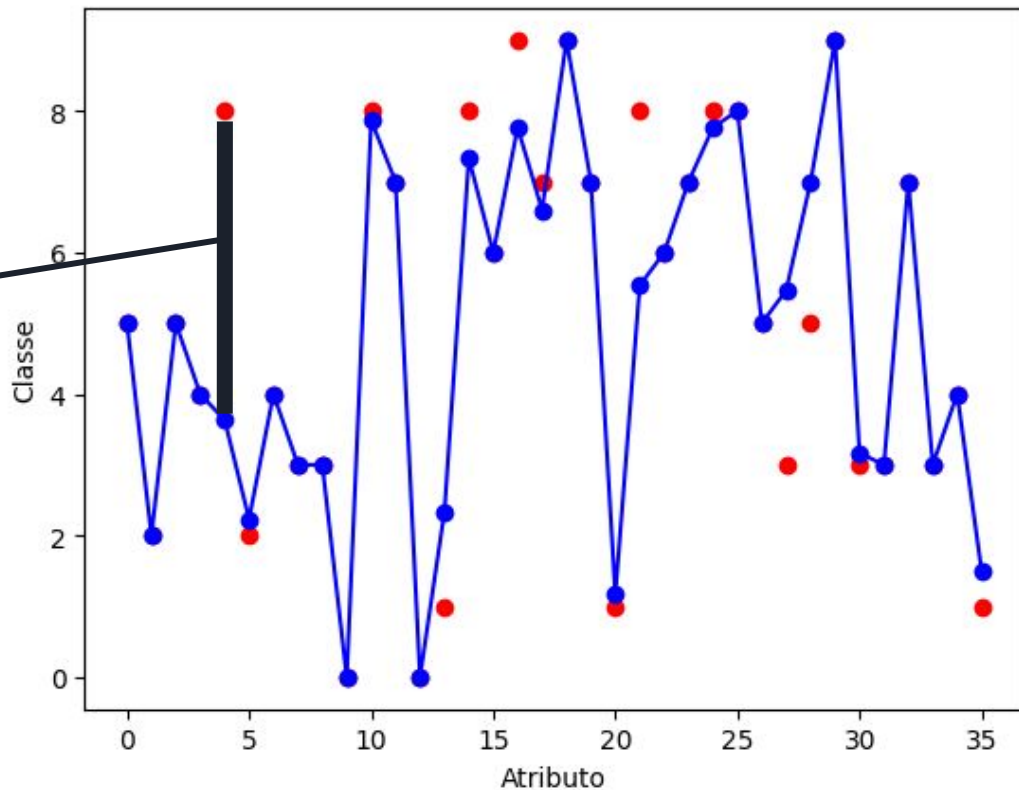
Valor previsto na posição i

Avaliação Regressor



Avaliação Regressor

Erro absoluto
de uma
amostra



Diferença entre MAE e MSE

- **Erro absoluto médio (MAE):**
 - Todos os erros possuem o mesmo peso, devido utilização do valor absoluto
 - Não é sensível a outliers
- **Erro quadrático médio (MSE):**
 - Erros maiores possuem maior peso, pois as diferenças são elevadas ao quadrado
 - Sensível a valores atípicos (outliers)



MAE e MSE com sklearn

- Calcular MAE:

- `mae = metrics.mean_absolute_error (previstos, teste_t)`
 - `mae`: erro absoluto médio
 - `previstos`: valores previstos pelo modelo
 - `teste_t`: alvos (classe) do conjunto de teste

- Calcular MSE:

- `mse = metrics.mean_squared_error (previstos, teste_t)`
 - `mse`: erro quadrático médio
 - `previstos`: valores previstos pelo modelo
 - `teste_t`: alvos (classe) do conjunto de teste



R^2 — Coeficiente de Determinação

- R^2 : mede quanto da variação da variável target é explicada pelo modelo

- $$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

- y_i : valor real
- $\hat{y}_i$: valor previsto
- $\bar{y}$: valor médio real

- Comparar o modelo treinado com uma previsão muito simples:
média de y

- $R^2 = 0$: modelo não apresenta vantagem em relação a simplesmente utilizar a média do target
- $R^2 < 0$: modelo pior que a média do target
- $R^2 > 0$: modelo melhor do que a média do target



RMSE — Root Mean Squared Error

- **RMSE:** mede o tamanho médio dos erros do modelo, dando maior peso aos erros grandes
 - $$RMSE = \sqrt{MSE}$$
 - RMSE fica na mesma unidade da variável target
 - **Exemplo:** target é salário em R\$, logo RMSE é salário em R\$
 - Quanto menor o RMSE, melhor
 - **RMSE vs MSE:**
 - **MSE:** mais conveniente para cálculos, otimização e comparação matemática
 - **RMSE:** mais conveniente para interpretação e apresentação dos resultados



Validação Cruzada (Cross-Validation)

- **Validação Cruzada:** Dados de treinamento são separados em grupos (dobras), com os quais modelos são treinados e validados
- **Validação Cruzada de K Dobras (K-Fold Cross Validation):** Conjunto de dados é dividido em K grupos (dobras) **aleatoriamente** onde:
 - Conjunto de treinamento: $K - 1$ grupos
 - Conjunto de validação: 1 grupo
 - Funcionamento:
 - Treinar o modelo com $K - 1$ grupos
 - Validar modelo com 1 grupo
 - Repete o processo de treinar e validar para cada um dos K grupos



Validação Cruzada de K-Dobras

	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
Split 1	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
Split 2	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
Split 3	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
Split 4	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
Split 5	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5

Treinamento	Teste

Validação Cruzada no sklearn

- Realizar validação cruzada

- `results = model_selection.cross_validate (ml_algo, atributos, target, cv=K, return_train_score=True, scoring=scoring)`
 - **results**: dicionário contendo o desempenho dado pela métrica escolhida, tempos de execução de treinamento e teste
 - **ml_algo**: instância de algoritmo de ML, exemplo KNN
 - **atributos**: Atributos do conjunto de dados
 - **target**: valores da variável alvo do conjunto de dados
 - **cv**: se é um inteiro, configura o número de K dobras. Se é uma instância de KFold, configura as dobras de acordo com KFold
 - **return_train_score**: retorna os resultados da avaliação com o conjunto de treinamento também
 - **scoring**: métrica utilizada. Exemplos: "accuracy", "neg_mean_absolute_error", "neg_mean_squared_error"



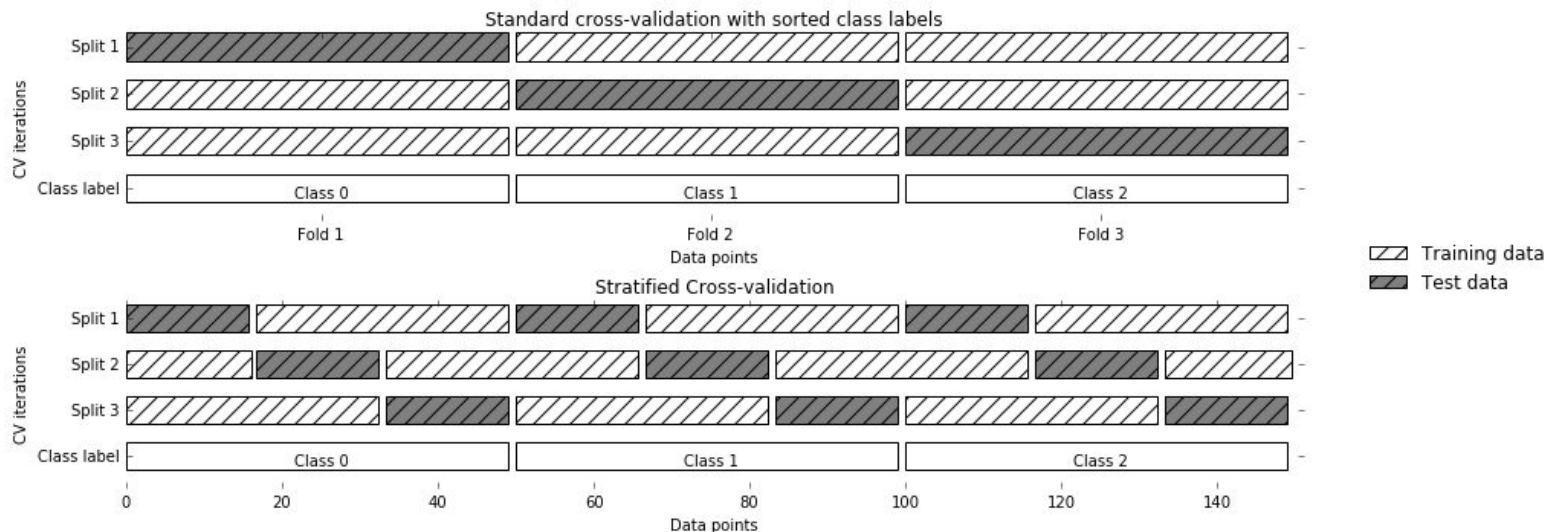
Validação Cruzada no sklearn

- **Realizar validação cruzada e obter apenas resultados**
 - `results = model_selection.cross_val_score (ml_algo, atributos, target, cv=K scoring=scoring)`
 - `results, ml_algo, atributos, target, cv, scoring`: mesmo que `cross_validate`
- **Realizar validação cruzada e obter previsões**
 - `model_selection.cross_val_predict(ml_algo, atributos, target, cv=K, scoring=scoring)`
 - Produz previsões para o fold de validação, porém retorna todas as previsões
 - **Exemplo para cv=2:**
 - Fold 1 → previsões das observações do fold 1
 - Fold 2 → previsões das observações do fold 2
 - Retorno: [Previsões Fold 1, Previsões Fold2]



Validação Cruzada de K-Dobras Estratificadas

- **Validação Cruzada de Leave-One-Out:** Proporções entre classes são as mesmas para cada dobra (fold)



Validação Cruzada Leave-One-Out

- **Validação Cruzada de Leave-One-Out:** Cada dobra (fold) contém apenas um único exemplo no conjunto de teste
 - **Grandes datasets:** pode apresentar alta complexidade computacional
 - **Pequenos datasets:** pode apresentar maior desempenho para pequenos datasets



	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
Exemplo 1	Exemplo 1				
Exemplo 2		Exemplo 2			
...	...	...	...	...	...
Exemplo N-1				ExemploN-1	
Exemplo N	Exemplo N	Exemplo N	Exemplo N	Exemplo N	Exemplo N

Validação Cruzada no sklearn

- Gerar dobras estratificadas

- `skfold = model_selection.StratifiedKFold (n_splits=5, shuffle=True)`
 - `skfold`: gerador python que permite criar os splits com um `for`
 - `n_splits`: número de splits
 - `shuffle`: se verdadeiro gera as dobras (folds) aleatoriamente

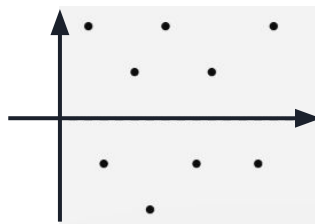
- Gerar dobras Leave-One-Out

- `loo = model_selection.LeaveOneOut ()`
 - `loo`: gerador python que permite criar os splits com um `for`

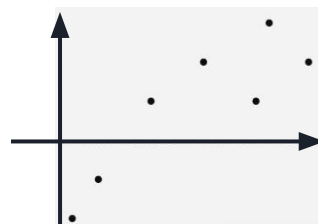


Análise de resíduos

- **Análise de resíduos:** responde se os erros do modelo parecem aleatórios ou existe algum padrão
 - Dois modelos podem possuir métricas semelhantes e apresentar comportamentos muito diferentes
 - **Homoscedasticidade:** modelo não apresenta a mesma variabilidade em todas as regiões dos dados
 - Espera-se que modelos não apresentem homoscedasticidade



Sem
Homoscedasticidade



Homoscedasticidade

- **Ferramentas:**
 - Residual Plot
 - Q-Q Plot

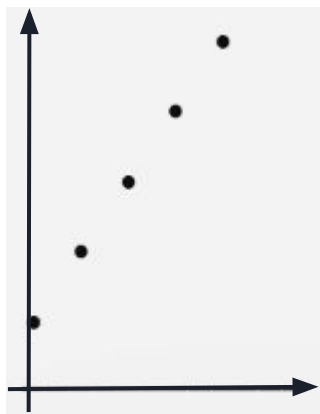
Residual Plot

- **Residual Plot:** é um gráfico utilizado para visualizar os erros
 - **Eixo-x:** valores previstos ($\hat{y}_i$)
 - **Eixo-y:** diferença dos valores previstos ($\hat{y}_i$) e valores reais (y_i)
- **Interpretação:**
 - **Pontos aleatórios ao redor de zero:** comportamento desejável; a relação linear pode ser razoável.
 - **Formato de curva:** pode existir uma relação não linear que o modelo não consegue representar.
 - **Formato de funil:** a variabilidade dos erros aumenta ou diminui; pode indicar heterocedasticidade.
 - **Pontos muito afastados:** podem indicar outliers ou observações difíceis de prever.
 - **Maioria acima ou abaixo de zero:** pode indicar tendência sistemática de subestimar ou superestimar.

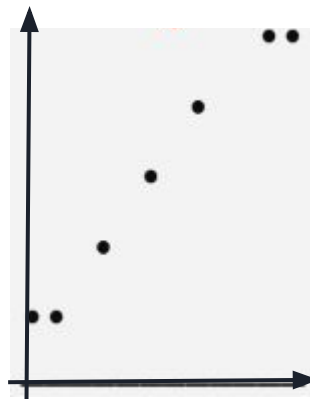


Q-Q Plot

- **Q-Q Plot - Quantile-Quantile Plot:** verificar se os resíduos apresentam uma distribuição aproximadamente normal
 - Eixo-x: resíduos observados
 - Eixo-y: amostras de distribuição normal
- **Exemplos:**



Aproximadamente
Normal



Não
Aproximadamente
Normal

- Ajuda a avaliar se as inferências estatísticas do modelo, como intervalos de confiança e testes de hipótese, são confiáveis

Residual Plot e Q-Q Plot

- Gerar gráfico Residual Plot com matplotlib:

- Scatter plot com eixo-x formado pelos valores previstos e eixo-y formado pelas diferenças das previsões

- **Exemplo:**

- ```
from sklearn.model_selection import cross_val_predict
import matplotlib.pyplot as plt
previstos_cv = cross_val_predict(modelo, X, y, cv=5)
residuos = y - previstos_cv
plt.scatter(previstos_cv, residuos)
plt.show()
```

- Gerar Gráfico Q-Q Plot com scipy.stats:

- `stats.probplot(valores, dist='norm', plot=plt)`

- **valores:** resíduos do modelo

- **Exemplo:**

- ```
from scipy import stats
import matplotlib.pyplot as plt
stats.probplot(residuos, dist="norm", plot=plt)
plt.title("Q-Q Plot dos Resíduos")
plt.show()
```